Tohannic : No Way Out

Proposition de projet

INF3001 – Simulation et Applications Interactives

INF3002 – Mouvements et Intelligence Artificielle

Maxime Dréan
drean.e2200832@etud.univ-ubs.fr

Mayssa Frikha
frikha.e2604618@etud.univ-ubs.fr

Mattéo Le Fur
le-fur.e2202500@etud.univ-ubs.fr

Paul Le Sager
le-sager.e2402846@etud.univ-ubs.fr

Objectif général. Concevoir un jeu d'évasion (*escape game*) en réalité mixte sur Meta Quest 3. Le joueur explore un lieu réel, manipule des indices virtuels sans contrôleurs et évite un monstre animé par capture de mouvement et piloté par une intelligence artificielle de navigation et de poursuite.

1. Intitulé du projet

Exploration en réalité mixte et poursuite par un monstre intelligent.

2. Concept et problématique

L'expérience prend place dans un environnement intérieur connu à l'avance. Le premier prototype se concentre sur une zone de jeu limitée et contrôlée, constituée d'une seule pièce. Le mode *passthrough* (affichage en direct des images filmées par les caméras du casque) permet de voir l'environnement physique, augmenté d'indices et d'un personnage virtuel. Le joueur se déplace réellement à allure contrôlée, sans manettes. L'ajout d'un couloir ou d'autres pièces pourra être envisagé après validation de cette première zone.

Le défi consiste à maintenir la correspondance entre les objets virtuels et les éléments physiques de la pièce pendant les déplacements du joueur. Le monstre doit également percevoir les obstacles, se déplacer dans l'espace accessible et réagir à la position du joueur, tout en préservant la sécurité des déplacements et les performances du casque autonome.

3. Expérience de jeu

Le joueur recherche trois indices virtuels répartis dans le parcours. Avec le *hand tracking* (suivi des mains par les caméras du casque), il saisit, déplace ou active des objets sans contrôleurs. Les indices servent à résoudre des énigmes successives et à débloquer une *sortie virtuelle*, sans interaction nécessaire avec une porte physique.

Nature des indices. Les indices seront des objets virtuels tridimensionnels, ancrés à des positions de la pièce et manipulables à la main. Une première proposition d'énigmes, à valider lors du prototypage, organise le parcours en trois étapes :

1. *Observation :* saisir une plaque virtuelle, l'orienter et relever un symbole visible sur l'une de ses faces.
2. *Manipulation :* faire tourner un disque gradué pour aligner ce symbole avec un repère et obtenir une nouvelle indication.
3. *Assemblage :* rapprocher deux fragments d'un même objet pour faire apparaître un code de déverrouillage.

Ce code sera saisi sur un clavier virtuel pour ouvrir une sortie également virtuelle. La logique des énigmes dépendra de la forme et de l'orientation des objets, et non uniquement de textes affichés dans l'espace. De courtes consignes pourront guider le joueur si une interaction est ambiguë.

Défis proposés au joueur. Le joueur doit trouver les indices dans un espace réel, comprendre comment les manipuler, mémoriser les informations découvertes et résoudre les énigmes tout en surveillant le monstre. La tension repose sur une alternance entre observation et dissimulation : lorsque le monstre l'aperçoit, le joueur doit rompre la ligne de vue derrière un obstacle puis profiter de la phase de recherche pour reprendre l'énigme. Le déplacement s'effectue à vitesse de marche, sans course ni contact avec les objets réels. La difficulté pourra être réglée par la vitesse du monstre, son champ de vision et la durée de sa recherche, sans imposer de chronomètre au prototype minimal.

La partie est gagnée lorsque l'énigme finale est résolue et que la sortie virtuelle est atteinte ; elle est perdue si le monstre intercepte le joueur. Le joueur conserve à tout moment une vue *passthrough* suffisante pour éviter les obstacles physiques.

Le premier prototype comportera une seule pièce, trois indices, un monstre et une condition de sortie. L'ajout d'un couloir ou d'autres pièces interviendra uniquement après validation de cette boucle de jeu.

4. Approche technique

4.1 Environnement et interactions (INF3001)

Unity et le *Meta XR Core SDK* (ensemble d'outils Meta pour développer sur Quest) seront utilisés pour afficher la scène en réalité mixte. La *Scene API* (accès aux surfaces et objets reconnus par le casque) et le Mixed Reality Utility Kit (MRUK) aideront à représenter les sols, murs et obstacles. Les ancrages spatiaux permettront de rattacher les objets virtuels à des positions physiques connues.

L'architecture fixe sera représentée en priorité à partir de la géométrie fournie par la Scene API et MRUK, afin de disposer d'une base directement exploitable pour le prototype. En complément, une

numérisation plus détaillée pourra être préparée avec ScanSpace, un outil de numérisation 3D dont le code est disponible sur GitHub, exploitant le LiDAR de l'iPhone ainsi qu'ARKit et RoomPlan. Le relevé pourra être exporté en GLB ou OBJ, puis nettoyé et simplifié dans Blender. Un recalage (mise en correspondance de deux systèmes de coordonnées) devra alors aligner ce maillage sur le repère spatial du Quest 3. Cette numérisation externe reste une amélioration : si sa calibration se révèle instable, la géométrie simplifiée issue de la scène Meta restera suffisante pour le prototype jouable.

Les meubles susceptibles d'être déplacés, notamment les tables ou les chaises utiles au parcours, ne seront pas intégrés définitivement au maillage fixe. Ils seront représentés par des volumes simples dans Unity et associés à des marqueurs visuels de type QR code. Après déplacement d'un meuble, la détection de son marqueur permettra de remettre à jour sa position et son orientation dans la scène virtuelle sans imposer un suivi continu. Cette fonction nécessitera un accès autorisé aux images des caméras via la Passthrough Camera API, le décodage du marqueur et une estimation de sa pose. Sa fiabilité devra être vérifiée sur le casque réel.

Géométrie et fabrication des objets. Les indices et accessoires virtuels seront des *maillages polygonaux* (sommets et faces) : les premiers essais utiliseront des primitives de Unity, puis des modèles légers créés ou corrigés dans *Blender* et importés, par exemple au format FBX. Le maillage visible (`Mesh Filter` et `Mesh Renderer`) sera distinct des volumes de collision simplifiés (*colliders*) et des zones de préhension. La plaque, le disque et les fragments d'assemblage pourront ainsi être manipulés sans simuler toute leur géométrie en physique. L'architecture fournie par MRUK prendra plutôt la forme de plans ou de volumes simplifiés ; seul un scan détaillé supplémentaire produirait un maillage de pièce plus complet.

Déplacement et déformation. Les indices ne se déformeront pas lors de leur manipulation : ils seront principalement translatés, tournés ou assemblés. Les meubles physiques, quant à eux, ne seront ni déformés ni modifiés : seul le modèle virtuel correspondant sera repositionné après lecture de son marqueur. Une transformation *visuelle* d'un meuble réel en élément fantastique pourra être expérimentée ultérieurement, sans changer la géométrie réellement occupée ni les contraintes de sécurité.

Le *Meta XR Interaction SDK* fournira les outils de manipulation par les mains. Des zones de sélection et d'activation adaptées aux gestes permettront d'interagir avec les indices. Les maillages utilisés pour les collisions seront simplifiés afin de limiter la charge de calcul.

4.2 Animation et intelligence artificielle (INF3002)

Le monstre sera représenté par un *maillage 3D déformable* associé à un squelette (*rig*). Le modèle sera préparé dans Blender : vérification de la topologie, placement des os et *weight painting* (poids attribués aux sommets pour déterminer l'influence des os). Les mouvements de l'équipe seront enregistrés avec FreeMoCap, solution libre de *motion capture* (capture de mouvement) sans marqueurs, au moyen de caméras préalablement calibrées. Les poses reconstruites seront nettoyées dans Blender, puis adaptées au squelette du personnage par *retargeting* (transfert d'animations entre squelettes différents). Dans Unity, l'*Animator* déclenchera les clips ; le *Skinned Mesh Renderer* déformera le maillage au niveau des articulations selon la pose des os (*skinning*). Il s'agit d'une déformation du maillage liée au mouvement des os : les membres fléchissent et les sommets suivent les articulations. Une transformation plus spectaculaire de la silhouette (allongement de membres, modification du visage, etc.) demanderait des *blend shapes* préparées dans Blender ou une animation spécifique. Elle sera considérée comme une extension visuelle facultative, distincte de l'animation

squelettique indispensable.

IA du monstre. Le système retenu est une *IA comportementale déterministe*, sans entraînement de réseau neuronal. Une machine à états finis (*FSM*) coordonnera les comportements suivants : patrouille → détection → poursuite → recherche → retour en patrouille. La perception utilisera la position du joueur, un cône de vision (distance et angle) et un test de ligne de vue par *raycast* contre les obstacles physiques reconstruits : un mur pourra donc interrompre la détection. En poursuite, le monstre mettra régulièrement à jour sa destination ; après perte de vue, il rejoindra la dernière position observée et y cherchera brièvement le joueur avant de reprendre sa patrouille. La capture sera déclenchée à proximité suffisante, sans collision physique avec le joueur.

La navigation reposera sur un NavMesh (surface indiquant les zones où le personnage peut marcher), parcouru par un agent de navigation Unity (`NavMeshAgent`) qui calcule un chemin contournant les obstacles. Les volumes des meubles suivis seront intégrés comme obstacles de navigation lorsque leur pose est connue. La partie « intelligence » correspond ici à la prise de décision, à la perception et au calcul de trajectoire ; une adaptation fondée sur l'historique du joueur reste une extension optionnelle et non un apprentissage automatique promis pour le prototype.

Exemple de décision. Si le joueur entre dans le cône de vision et qu'aucun mur ne le masque, l'IA passe de la patrouille à la poursuite. S'il se cache derrière une table dont le volume est reconnu comme occultant, le monstre cesse de recevoir sa position actuelle, rejoint la dernière position visible puis explore brièvement les alentours. Le joueur peut alors s'éloigner ou reprendre une énigme. L'IA ne connaîtra donc pas la position du joueur en permanence ; l'angle, la portée, les délais et le périmètre de recherche seront réglables pour obtenir une difficulté jouable dans une seule pièce.

4.3 Défis techniques et limites du prototype

— *Recalage spatial :* éviter que les indices et les collisions dérivent par rapport aux murs réels. Solution envisagée : une pièce calibrée, des ancrages spatiaux et des essais de retour au point de départ ; le scan détaillé est secondaire.

— *Mobilier mobile et marqueurs :* reconnaître le QR code malgré les changements de distance, d'angle ou de luminosité, puis reconstruire la pose du meuble. Solution : tests précoces de la Passthrough Camera API, marqueurs de dimensions connues et vérification après chaque déplacement ; si nécessaire, recours à un recalage manuel pour les essais.

— *Suivi des mains et énigmes :* gérer les occlusions des doigts, les gestes imprécis et la perte temporaire du suivi. Solution : objets suffisamment grands, interactions tolérantes et possibilité de réessayer sans bloquer la progression.

— *Animation du monstre :* éviter les poses irréalistes produites par la capture et les déformations incorrectes aux articulations. Solution : nettoyage des clips, réglage des poids du squelette dans Blender et transitions d'animation progressives dans Unity.

— *Navigation et réactivité :* produire des trajectoires crédibles lorsque les obstacles changent de place, sans recalcul coûteux à chaque image. Solution : NavMesh de la pièce, mise à jour ciblée des obstacles et fréquence contrôlée de recherche de chemin.

— *Équilibrage des poursuites :* éviter un monstre omniscient ou, à l'inverse, incapable de retrouver le joueur dans une seule pièce. Solution : champ de vision et temporisations paramétrables, essais de perte puis de reprise de cible et ajustement du temps consacré aux énigmes.

— *Sécurité et performances :* préserver la fluidité du Quest 3 et éviter qu'une poursuite incite à

courir ou à heurter un meuble. Solution : aire réelle dégagée, déplacements lents, distances de sécurité, maillages simplifiés et mesures en situation réelle.

Exemples techniques :

— First Hand : référence pour le suivi et la manipulation avec les mains.
— Project Phanto : exemple d'interaction virtuelle avec la géométrie d'une pièce réelle.

5. Fonctionnalités et priorités

Les exigences sont réparties selon leur contribution à une démonstration complète. Les fonctionnalités de niveau Indispensable forment le parcours minimal : une seule défaillance bloquante empêche de conclure une partie. Les fonctionnalités de niveau Importante renforcent la fidélité et l'étendue du projet, mais ne doivent pas retarder ce parcours. Les fonctionnalités de niveau Optionnelle sont réservées au temps restant après stabilisation.

Chaque ligne indique un comportement observable et un critère de validation, afin de distinguer l'implémentation d'un simple essai technique.

Importance	**Fonctionnalité attendue**	**Critère de validation**
Indispensable	Lancer l'application sur Quest 3 en *passthrough* et récupérer les surfaces nécessaires du lieu (sols, murs, obstacles).	Affichage stable en conditions réelles et repérage des obstacles.
Indispensable	Calibrer les positions des indices dans une seule pièce et les maintenir lors des déplacements et changements d'orientation.	Trois objets restent à leurs emplacements après un aller-retour.
Indispensable	Repositionner dans la scène virtuelle un meuble réel identifié par QR code après son déplacement.	Un meuble déplacé puis reposé retrouve une position et une orientation cohérentes dans Unity.
Indispensable	Reconnaître les deux mains et permettre de sélectionner, prendre, relâcher et activer chaque indice sans manettes.	Trois interactions enchaînées sans contrôleur.
Indispensable	Gérer trois indices 3D manipulables (examen, rotation, assemblage), la saisie d'un code sur un clavier virtuel, la sortie et les conditions de victoire et de défaite.	Résoudre les trois indices puis gagner ; tester aussi une partie perdue.
Indispensable	Faire naviguer le monstre entre des points de patrouille en contournant les obstacles et sans traverser les murs.	Parcours préétabli réalisé sans collision bloquante.
Indispensable	Détecter le joueur dans un cône de vision, vérifier la ligne de vue, poursuivre puis rechercher la dernière position connue.	Tests : joueur visible, caché et perdu de vue.

Importance	Fonctionnalité attendue	Critère de validation
Indispensable	Préparer le squelette et les poids du maillage du monstre, puis intégrer au moins deux animations issues de FreeMoCap selon les états de l'IA.	Marche et course visibles sur le personnage, sans déformation articulaire bloquante.
Indispensable	Définir une zone physique sûre, gérer les pertes de suivi et contrôler les performances sur casque autonome.	Parcours testé sans collision réelle ; mesures de fluidité relevées.

5.1 Améliorations et extensions

Les fonctionnalités suivantes ne sont abordées qu'après validation du parcours minimal. Les améliorations ont pour but d'élargir le lieu exploré et de rendre les interactions plus réalistes. Les extensions ne sont développées que si les tests d'intégration sont concluants.

Importance	Fonctionnalité attendue	Critère de validation
Importante	Scanner la pièce dans ScanSpace, exporter un maillage, le corriger dans Blender et l'aligner avec le repère Quest.	Sol, murs et principaux obstacles recalés sur les mêmes repères physiques.
Importante	Ajouter un couloir puis, si pertinent, étendre la scène à plusieurs pièces en conservant la cohérence des indices et de la navigation entre les zones.	Déplacement aller-retour entre les zones sans décalage gênant ni blocage.
Importante	Ajouter l'occultation des objets virtuels par les murs et un réglage plus précis des volumes de collision.	Un indice disparaît correctement derrière un obstacle.
Importante	Ajouter des animations de recherche et d'attaque, des transitions fluides et des retours sonores ou visuels lisibles.	États perceptibles sans rupture manifeste d'animation.
Optionnelle	Adapter la zone de recherche du monstre selon les derniers déplacements observés du joueur.	Deux comportements distincts comparés sur un même scénario.
Optionnelle	Déformer ponctuellement la silhouette du monstre (visage ou membres) avec des *blend shapes* préparées dans Blender.	Une transformation déclenchée par un état est visible sans perturber les animations de marche.
Optionnelle	Varier les emplacements des indices entre deux parties sans compromettre la résolution des énigmes.	Deux configurations jouables et validées.

Importance	Fonctionnalité attendue	Critère de validation
Optionnelle	Ajouter des objets virtuels interactifs répartis dans la zone de jeu, avec des effets possibles sur la progression ou sur le comportement du monstre.	Au moins un objet peut être manipulé et déclenche un effet de jeu identifiable, par exemple attirer, distraire ou ralentir le monstre.
Optionnelle	Transformer visuellement certains meubles réels suivis par QR code en éléments du décor virtuel, par exemple une table représentée comme un mur ou une cloison.	Le décor virtuel reste aligné avec le meuble réel et conserve une zone physique cohérente pour les déplacements du joueur.
Optionnelle	Proposer une ambiance virtuelle sombre afin de limiter volontairement la visibilité et d'augmenter la tension pendant l'exploration.	Le parcours et les indices restent utilisables malgré la réduction de visibilité, sans compromettre la sécurité du joueur.
Optionnelle	Ajouter une mécanique de lampe ou de source de lumière, disponible dès le départ ou à retrouver dans la zone de jeu.	La lampe améliore localement la visibilité et peut être intégrée à une courte étape de progression.

5.2 Scénarios de validation

La validation ne se limitera pas à l'ouverture de la scène Unity. Des essais reproductibles seront effectués dans l'environnement physique retenu, en conservant les conditions de test et les anomalies rencontrées.

— *Alignement spatial :* poser les indices à trois emplacements de référence, effectuer un trajet complet et vérifier leur position à l'arrivée. Recommencer après une nouvelle session du casque.

— *Manipulation et progression :* examiner la plaque, faire tourner le disque, assembler les fragments puis saisir le code ; vérifier qu'une perte temporaire de suivi des mains ne rend pas l'énigme impossible.

— *Comportement du monstre :* tester séparément les situations où le joueur est visible, masqué par un mur, puis perdu de vue ; vérifier qu'il recherche la dernière position vue, sans suivre instantanément le joueur à travers un obstacle.

— *Maillage animé :* observer la marche et la course au niveau des genoux, coudes et épaules pour repérer des déformations anormales ou des ruptures d'animation.

— *Parcours complet :* terminer au moins une partie gagnée et une partie perdue, puis relever la fluidité et les incidents rencontrés durant les déplacements réels.

L'ordre de réalisation peut être ajusté en cas de difficulté technique. En revanche, aucune extension optionnelle ne doit compromettre les fonctionnalités indispensables.

6. Roadmap

Le développement est organisé en deux MVP successifs, correspondant aux deux unités d'enseignement. Le MVP 2 dépend directement du MVP 1 : avant d'ajouter un monstre capable de naviguer et de réagir au joueur, il faut disposer d'un environnement réel correctement représenté, calibré et exploitable dans Unity. Les phases sont ordonnées par dépendances techniques et ne sont pas associées à des échéances figées.

MVP 1 – INF3001 : environnement MR jouable

L'objectif du premier MVP est d'obtenir une petite boucle d'escape game stable sur Quest 3, sans dépendre du monstre.

Phase 1 – Mise en place Unity et Quest 3

Objectifs. Configurer le projet Unity pour le Quest 3 standalone, intégrer OpenXR et les SDK Meta nécessaires, obtenir un affichage *passthrough*, déployer une première application sur le casque et vérifier le suivi des mains pour les interactions prévues.

Livrable : application de test fonctionnelle sur Quest 3 avec *passthrough* et détection des mains.

Phase 2 – Numérisation et calibration de l'environnement

Objectifs. Choisir une seule pièce comme zone de test, puis récupérer une représentation simple de son architecture fixe avec la Scene API et MRUK. Définir un repère physique de référence et vérifier que les murs et les obstacles restent suffisamment cohérents pendant les déplacements. Si le temps et la stabilité du recalage le permettent, un scan plus détaillé réalisé avec ScanSpace pourra ensuite remplacer ou compléter cette géométrie simplifiée.

Livrable : représentation géométrique simple de la zone de jeu, alignée avec l'environnement réel et vérifiée sur casque ; le scan détaillé reste optionnel à ce stade.

Phase 3 – Mobilier amovible

Objectifs. Représenter quelques meubles mobiles par des volumes simples dans Unity et leur associer des QR codes. La détection d'un code doit permettre de retrouver la position et l'orientation du meuble, puis de mettre à jour son volume de collision et, lorsque nécessaire, son obstacle de navigation. Le système vise en priorité les meubles déplacés puis reposés, sans imposer un suivi continu à haute fréquence.

Livrable : au moins un meuble réel déplacé puis correctement repositionné dans la scène virtuelle grâce à son QR code.

Phase 4 – Indices, interactions et progression

Objectifs. Placer trois indices sous forme de maillages virtuels dans l'espace calibré et prototyper les manipulations proposées : inspection d'une plaque, rotation d'un disque et assemblage de fragments. Ajouter la saisie du code final, la validation de la sortie virtuelle et la sauvegarde des états de progression. Vérifier qu'une perte temporaire de suivi ou une interaction répétée ne bloque pas la partie ; simplifier une énigme si son geste se révèle peu fiable sur Quest 3.

Livrable : boucle d'escape game jouable sans manettes et sans monstre, depuis l'entrée dans la zone jusqu'à la condition de victoire.

Validation du MVP 1. Le joueur doit pouvoir entrer dans la zone réelle, retrouver les éléments virtuels aux positions prévues, interagir avec les indices et terminer une courte séquence de jeu. Le mobilier suivi ne doit pas créer d'incohérence majeure entre l'espace physique et la scène Unity.

MVP 2 – INF3002 : monstre animé et intelligent

Le second MVP réutilise directement l'environnement, les obstacles et la boucle de progression validés pendant INF3001.

Phase 5 – Graphe d'états et préparation de la motion capture

Objectifs. Définir avant l'enregistrement un graphe complet des états du monstre et de leurs transitions : patrouille, détection, poursuite, perte de la cible, recherche de la dernière position connue, retour en patrouille et capture. À partir de ce graphe, établir la liste des animations et transitions réellement nécessaires.

Livrable : graphe d'états validé et liste des clips de motion capture à enregistrer.

Phase 6 – Capture et intégration des animations

Objectifs. Préparer dans Blender un monstre avec maillage, squelette et poids des sommets (*skinning*). Enregistrer avec FreeMoCap les mouvements définis à la phase précédente, nettoyer les captures, les transférer au squelette du monstre et préparer les transitions et le *blending* dans Unity. La marche et la course constituent le minimum attendu ; les animations de recherche et d'attaque seront ajoutées si leur qualité est suffisante. Vérifier les déformations aux articulations lors des changements d'état.

Livrable : au moins deux animations issues de captures réelles et correctement déclenchées dans Unity.

Phase 7 – Navigation, perception et comportement

Objectifs. Préparer le NavMesh à partir de l'architecture fixe, intégrer les meubles suivis par QR comme obstacles et faire naviguer le monstre dans les zones accessibles avec un `NavMeshAgent`. Ajouter le cône de vision, la distance maximale, un raycast de ligne de vue et la mémorisation de la dernière position connue du joueur. Relier ces informations à la machine à états (patrouille, poursuite, recherche, capture), en distinguant clairement cette IA à règles d'un système entraîné par apprentissage automatique.

Livrable : monstre capable de patrouiller, détecter, poursuivre, perdre puis rechercher le joueur sans traverser les murs ni les obstacles reconnus.

Phase 8 – Intégration, essais et stabilisation

Objectifs. Relier le monstre à la boucle de jeu du MVP 1, gérer la capture du joueur et la défaite, puis tester l'ensemble sur Quest 3. Vérifier en priorité l'alignement spatial, les collisions, les pertes de suivi, la lisibilité des interactions, la navigation et la fluidité. Les erreurs empêchant de terminer une partie sont corrigées avant toute extension.

Livrable : prototype complet dans lequel le joueur peut résoudre les indices tout en étant poursuivi par un monstre animé et cohérent avec l'environnement réel.

Validation du MVP 2. Une partie complète doit pouvoir être gagnée ou perdue sur casque, avec navigation du monstre, perception du joueur, transitions d'animation et respect de la géométrie fixe et du mobilier détecté.

Phase 9 – Évaluation et restitution

Objectifs. Faire tester le prototype à des personnes extérieures à l'équipe, relever les difficultés de prise en main et vérifier les scénarios de victoire et de défaite. Consolider le projet Unity, les ressources 3D, le protocole de calibration, les captures de mouvement et la documentation des choix techniques et des limites rencontrées.

Livrable : démonstration sur Quest 3, code et ressources du projet, procédure de lancement et bilan critique des résultats.

Coordination et critères de passage

Certaines tâches pourront progresser en parallèle, mais l'ordre des dépendances doit être conservé. En particulier, la capture de mouvement n'est lancée qu'après définition du graphe d'états, et le comportement complet du monstre n'est intégré qu'une fois le MVP 1 suffisamment stable. Les tests sont réalisés régulièrement sur le Quest 3 plutôt que dans le seul éditeur Unity.

Pour passer d'une phase à la suivante, l'équipe doit disposer d'un livrable exécutable ou vérifiable et d'une liste claire des points encore ouverts. Les fonctionnalités indispensables sont stabilisées avant les extensions optionnelles. Les ajouts tels que des objets virtuels supplémentaires, la transformation visuelle du mobilier ou une ambiance sombre avec une mécanique de lampe ne sont envisagés qu'après validation des deux MVP.